Minomaly: A Subgraph Mining Framework for Unsupervised and Interpretable Anomaly Detection in Graphs

Ahmed Youcef Benhalima[1], Seif-Eddine Benkabou[2*], Amin Mesmoudi[2], Allel Hadjali[1]

[1]ISAE-ENSMA, France.
[2]Université de Poitiers, France.

Abstract

Detecting structural anomalies in graphs is crucial to uncover unexpected patterns and mitigate risks in fields such as cybersecurity, social network analysis, and biological research. Existing methods, which often rely on machine learning approaches, struggle to balance detection effectiveness, scalability, and interpretability. In this paper, we introduce Minomaly, an unsupervised framework that combines subgraph mining with Graph Neural Networks (GNNs) to detect rare node neighborhoods indicative of structural anomalies. By embedding subgraphs into an Order Embedding Space (OES), Minomaly efficiently estimates subgraph frequencies and employs a greedy search algorithm to expand infrequent node structures, with interpretable and controllable hyperparameters. This approach enables the detection of anomalies in a way that is both interpretable and computationally efficient. Experiments on well-established benchmark datasets demonstrate that Minomaly consistently outperforms the state-of-the-art methods in detection quality and scalability, handling large graphs without memory overhead issues. The implementation of Minomaly along with the associated experimental setup, is available at https://github.com/gbay7/minomaly.

Keywords: Graph Anomaly Detection, Unsupervised Learning, Graph Neural Networks, Order Embedding

1 Introduction

Graph a Anomaly Detection (GAD) plays a crucial role in uncovering hidden insights and mitigating risks in various domains. For example, in cybersecurity, detecting anomalous network traffic patterns can help identify potential intrusions or malicious activities. In social network analysis, identifying anomalous behaviors can assist in detecting fake accounts, spam, or unusual communication patterns. Similarly, in biological networks, anomaly detection can highlight proteins or genes with unusual interactions, shedding light on disease mechanisms or novel biological processes [**? ? ?**]. In financial transaction networks, the presence of cliques might indicate unusual or suspicious activity. For instance, if multiple accounts are found to be tightly connected through transactions (forming a clique), it could suggest fraudulent activity such as money laundering or collusion between accounts (See Figure 1).

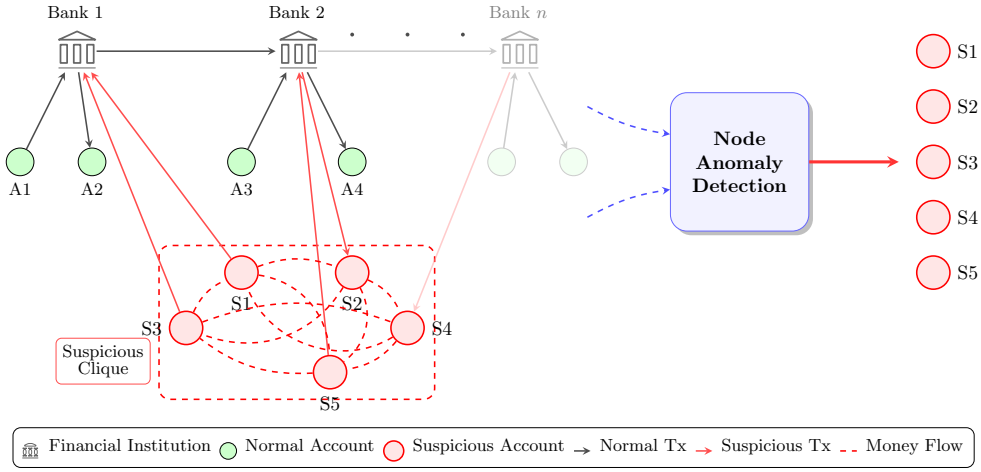


Fig. 1: Financial fraud detection using Graph Anomaly Detection. The figure illustrates how suspicious money flow patterns form cliques among accounts (S1-S5), suggesting potential money laundering activities. A GAD algorithm processes the financial network to identify these unusual structural transaction patterns, enabling financial institutions to flag suspicious accounts for further investigation.

Regardless of the specific application area (e.g., fraud detection, security monitoring), unsupervised node anomaly detection has attracted considerable research attention due to its broad applicability [**? ?**]. In graph data, nodes are interconnected through edges to form intricate patterns and structural relationships. These patterns represent interactions, connections, or relationships between entities and often encode meaningful insights relevant to various analytical tasks. The structural arrangement and frequency of these connections can reveal typical behaviors, communities, or clusters, making them critical for identifying deviations indicative of anomalies. Nodes with attribute vectors that deviate considerably from the global attribute norm are

often considered global anomalies. While node attributes provide valuable information, the edges between nodes offer additional insights. If a node belongs to an unusual connection pattern or neighborhood, all nodes within this structure are collectively anomalous and called structural anomalies. When node attributes are considered within this abnormal structure, these anomalies are referred to as contextual anomalies [**?** **?**].

Our primary focus, however, is on **structural anomalies**. While existing methods have advanced in detecting these anomalies, they often lack generality and interpretability. For instance, BOND [**?**] considers structural anomalies as densely connected nodes. However, in some real-world geometric graphs, dense connections are common, making this assumption inadequate. Other methods rely on specific motif distributions [**?** **?** **?**], which limits their applicability to diverse real-world contexts such as fraud detection because they do not generalize to all motifs that may exist. Existing unsupervised deep learning techniques often lack solid explanations for their results, limiting their interpretability [**?**]. Traditionally, frequent pattern outlier detection methods provided interpretable results but were computationally expensive due to the need to enumerate patterns and match similar ones [**?** **?** **?**]. This computational burden arises because the problem reduces to the subgraph isomorphism problem, which is NP-complete. In these methods, frequency serves as a measure of abnormality because rare structures are more likely to be anomalous.

To address these limitations, we propose **Minomaly**, an unsupervised, interpretable, and scalable framework for structural anomaly detection in graphs. Our approach proceeds in two distinct steps:

- ***Order Embedding Space (OES) construction*** — Minomaly combines subgraph mining with GNN-based order embeddings to encode graph neighborhoods efficiently. This step allows for accurate frequency estimation of subgraphs.
- ***Anomalous structures search*** — Using the geometrical properties of the OES, Minomaly employs a novel greedy search algorithm to identify rare and unusual node structures, allowing interpretable anomaly detection.

Our contributions can be summarized as follows:

- A new formalization of structural anomaly detection in graphs using subgraph mining. A proof of the correctness of the counting measure used to enumerate anomalous subgraphs, based on the anti-monotonicity property, is provided. Additionally, a generalized definition of structural node anomaly detection is proposed to support anomaly detection across multiple contexts.
- A novel application of order embeddings to graph anomaly detection. Minomaly leverages GNN order embeddings to efficiently encode graph structures.
- A new optimization technique in anomalous structures search by exploiting the geometrical properties of the OES which significantly reduces the number of nodes that need to be verified during the search for anomalous structures.
- A novel, computationally efficient greedy search algorithm for identifying and interpreting unusual structures in graphs, enabling interpretable GAD.
- An in-depth experimental study on widely used benchmark datasets to validate Minomaly's performance.

The rest of this paper is organized as follows: Section 2 introduces the related works. Section 3 presents the Minomaly framework in detail while formalizing the problem of node anomaly detection in graphs. Section 4 includes our performance study and experimental results compared with existing approaches. Section 5 highlights key insights from our experiments and discusses their implications. Finally, Section 6 concludes the paper and outlines future research directions.

2 Related Work

Anomaly Detection (AD) research spans diverse data modalities, each presenting unique challenges. Classical methods for tabular data rely on statistical, distance-based, and isolation approaches [**? ? ? ?**], while time series analysis employs techniques designed to capture temporal dependencies [**? ? ? ?**]. Recent deep learning advancements have further refined anomaly detection across these domains [**? ? ?**]. In contrast, graph-structured data introduces additional complexity, as anomalies may emerge within individual nodes, across relationships, or through unexpected structural patterns. Unlike tabular data, where observations are independent, graphs require specialized methods that simultaneously model structural connections and node attributes to identify irregularities effectively.

Graph Anomaly Detection (GAD) has been extensively studied, employing a range of methods from traditional algorithms to advanced deep learning approaches [**?**]. These techniques aim to identify anomalous patterns in graph-structured data, which can be critical for applications such as fraud detection, cybersecurity, and biological network analysis. Traditional methods for graph anomaly detection can be divided into two main categories: machine learning-based approaches and pattern mining techniques.

Machine learning methods often rely on matrix decomposition and clustering algorithms to detect anomalies. Radar (residual analysis) [**?**] identifies anomalies by examining residuals between node attributes and their network coherence, scoring outliers based on reconstruction residual norms. ANOMALOUS (CUR decomposition) [**?**] applies CUR decomposition [**?**] and residual analysis to jointly detect anomalies and select relevant node attributes. Clustering-based approaches, such as SCAN (structural clustering) [**?**], identify irregularities by grouping similar nodes and flagging those that deviate from expected patterns. While these methods provide interpretable results, they often suffer from performance limitations when applied to complex graph structures, particularly in large-scale graphs where scalability remains a challenge. Matrix decomposition operations face high computational complexity in both dense and sparse adjacency matrices, with memory requirements growing quadratically with the number of nodes. Similarly, clustering-based methods require computing pairwise similarities across the entire graph, which becomes prohibitive for graphs with millions of nodes.

Pattern mining techniques provide an alternative perspective for detecting anomalies in graphs [**? ?**]. Graph pattern mining aims to discover recurring substructures within graph data, ranging from frequent subgraph patterns to specific motifs and topological features. Traditional exact methods, such as GSpan [**?**] and GASTON [**?**

], systematically enumerate all frequent patterns through exhaustive search strategies, while approximate approaches like Motivo [**?**] trade completeness for efficiency. However, these methods exhibit exponential complexity with respect to subgraph pattern size, severely limiting their applicability to large-scale graphs. To address these scalability challenges, deep learning approaches leveraging Graph Neural Networks (GNNs) have emerged, including FlowSC [**?**], SPMiner [**?**], and D2Match [**?**], which offer improved efficiency through learned representations. Among these, SPMiner relies on order embedding [**? ?**] to count frequent graph patterns, yet it lacks accuracy in predicting graph-level frequency as this was its primary focus to estimate it, and the correctness of frequency estimates based on exact motif matching was not proven.

Building upon pattern mining foundations, PANG [**?**] focuses on detecting unexpected subgraph patterns indicative of anomalies. It converts graphs into vector representations using discriminative subgraph patterns extracted through frequent graph mining algorithms such as GSpan, enabling interpretable anomaly detection via classification techniques. GUIDE [**?**] uses a graph counting algorithm focused on specific three-node and four-node motifs. However, as these approaches rely on traditional graph mining techniques, they encounter significant computational challenges due to the combinatorial explosion of subgraph patterns, which limits their scalability.

Recent advances in deep learning have significantly improved graph anomaly detection [**? ? ? ?**]. In this context, Graph neural networks (GNNs) have demonstrated exceptional performance in various graph mining tasks, making them well-suited for detecting anomalous nodes [**?**]. These methods leverage representation learning to capture intricate structural dependencies and complex attribute relationships, allowing for more precise node anomaly detection.

Reconstruction-based anomaly detection has gained popularity among deep learning techniques. These models aim to learn the global behavior of node information but often struggle with detecting rare patterns. DOMINANT [**?**] employs a GCN encoder with separate decoders for structural and attribute reconstruction, while AnomalyDAE [**?**] uses two independent autoencoders (AEs) to combine reconstruction losses for anomaly scoring. DONE [**?**] introduces MLP-based autoencoders for structural and attribute reconstruction, whereas AdONE [**?**] enhances this method with adversarial learning. Similarly, CONAD [**?**] leverages graph augmentation and contrastive learning, incorporating human prior knowledge of structural and contextual anomalies. However, reconstruction-based approaches may generate high reconstruction errors for anomalous nodes, as balancing the learning of normal and abnormal patterns remains challenging and poorly controlled, which makes them difficult to interpret due to the black-box nature of deep neural network representations and the lack of direct semantic meaning in reconstruction errors [**?**].

Generative models offer an alternative to reconstruction-based approaches by leveraging synthetic data to improve anomaly detection. GAAN [**?**] employs a generator to create synthetic graph nodes, an encoder to map nodes into latent space, and a discriminator to differentiate between real and synthetic nodes. However, these models assume that anomalous nodes resemble synthetic ones, which can reduce their interpretability and generalization capabilities.

Interpreting the predictions of graph anomaly detection models is essential for real-world applications. Despite increasing research efforts in explainable artificial intelligence (XAI), most existing work focuses on general GNN explainability methods [**? ? ?**]. While these methods provide insights into model behavior and hyperparameter tuning, they are not specifically tailored for anomaly detection. Only a few recent studies [**? ? ? ? ? ? ?**] have addressed the explainability of GAD results, highlighting a significant gap in interpretable anomaly detection techniques.

To address the limitations of existing graph anomaly detection techniques, we introduce Minomaly, a novel framework that redefines anomaly identification by optimally balancing performance, scalability, and interpretability. Unlike conventional methods, Minomaly integrates subgraph mining with GNN-based order embeddings, enabling more precise detection and a deeper understanding of rare structural anomalies in graphs. Specifically, it applies order embeddings to node-level and subgraph-level anomaly detection, leveraging infrequent patterns to identify structural anomalies with interpretable search mechanisms. While the majority of state-of-the-art methods focus solely on providing an anomaly score indicating that a node is structurally anomalous without revealing the specific structure responsible for the anomaly, Minomaly delivers essential interpretable results by explicitly identifying the anomalous structure itself.

3 Minomaly Framework

In this section, we present Minomaly, an approach that uses a GNN for detecting structural anomalies in graphs. Through the GNN, Minomaly embeds graph structures into an Order Embedding Space (OES), where the embeddings provide an approximate frequency count of node structural neighborhoods to identify rare patterns indicative of node anomalies. These detected anomalous structures are identified through a greedy search method, which expands infrequent neighboring graphs using OES, showcasing the interpretability of Minomaly.

For efficiency and effectiveness, a heuristic search is applied to reduce the anomaly search space by selecting potential nodes from the OES that produce anomalous patterns, rather than verifying the normality of all graph nodes.

Overall, the Minomaly framework operates in two steps: **Order Embedding Space Construction** and **Anomalous Structures Search** (See Figure 2).

We formally introduce the foundational graph-theoretic concepts of static graphs (representing the data), induced subgraphs and isomorphism, which serve as the basis for our order-based frequency measure, relying on subgraph matching to count rare node structures.

Definition 1 (Static Directed Graph)**.** *A static directed graph $G = (V, E)$ consists of a fixed set of nodes V, and edges $E \subseteq V \times V$ which represent the connections between nodes. The order of G is its number of nodes $|V|$, and its size is its number of edges $|E|$.*

Definition 2 (Induced Subgraph)**.** *Let $S \subseteq V$. The induced subgraph $G[S]$ in G by S is the graph $G[S] = (S, E_S)$, where $E_S = \{(u, v) \in E \mid u, v \in S\}$, the set of edges from G that connect two nodes in S. We write $G' \subseteq G$ if G' is subgraph of G, and we denote $\boldsymbol{\mathcal{G}(G)}$ the set of subgraphs of G.*

Definition 3 (Isomorphism)**.** *Let $G[S], G[T] \in \mathcal{G}(G)$. We say $G[S]$ and $G[T]$ are isomorphic, denoted $G[S] \cong G[T]$, if and only if:*

$$\begin{cases} \exists f : S \to T, \ f \text{ is bijective}, \\ \forall u_1, u_2 \in S : (u_1, u_2) \in E_S \iff (f(u_1), f(u_2)) \in E_T. \end{cases}$$

To provide a general definition of structural anomalies, beyond those based on specific assumptions [**? ? ? ?**], we propose our concept of a node structure:

Definition 4 (Node Structure)**.** *Let $u \in V$. The **structure** or **neighborhood graph** of the node u is the connected subgraph induced by the set of nodes that are reachable from u, including u itself.*

- *The term reachable refers to the nodes that can be reached by traversing edges starting from u.*
- *This subgraph has a limited size, determined by the size of the neighborhood within a defined distance or number of nodes.*
- *Ego graphs refer to one-hop neighbors, which is more restrictive.*

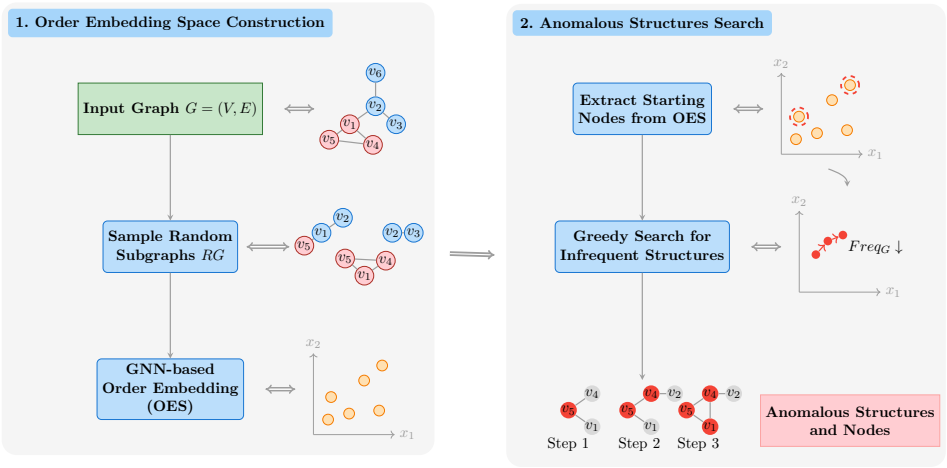


Fig. 2: Minomaly Framework

3.1 Order Embedding

Here, we introduce our method for estimating the **frequency** of small subgraphs to identify anomalous node structures based on their rarity. This approach is motivated by the observation that in most real-world graphs, anomalous behaviors manifest as rare structural patterns that deviate significantly from the majority of normal neighborhoods [**?**]. By focusing on small subgraphs, we capture local structural deviations while maintaining computational efficiency. Under the assumption that rare structures are more likely to be anomalous [**?**], we leverage subgraph frequency as a fundamental measure of abnormality.

Intuitively, the frequency of a subgraph refers to the number of subgraphs that match it isomorphically. Finding these matches reduces to the NP-complete subgraph isomorphism problem [**?**]. To address this, we first tackle the **subgraph matching** problem by training a GNN to determine whether a given graph is a subgraph of another.

The subgraph matching problem naturally exhibits a partial order structure: if $G_1 \subseteq G_2$, then any graph containing G_2 must also contain G_1. Order embeddings provide an ideal framework for capturing this hierarchical containment relationship, allowing us to efficiently approximate the NP-complete subgraph isomorphism problem through learned continuous representations. By preserving the partial order in the embedding space, order embeddings enable efficient frequency estimation via geometric operations, transforming the discrete combinatorial problem into a continuous optimization task. Given these partial order properties of subgraph matching, we use **order embeddings** to map these graphs and capture these properties. We formalize this encoding through our proposed order embedding function:

Definition 5 (Order Embedding Function)**.** *An **order embedding** function for graphs, denoted as $\phi : \mathcal{G}(G) \to \mathbb{R}^d$, is an injective function that maps a subgraph of G into a d-dimensional embedding space such that for two subgraphs $G_1, G_2 \in \mathcal{G}(G)$:*

$$G_1 \subsetneq G_2 \iff \phi(G_1) \preceq \phi(G_2), \quad \text{such that } G_1 \subsetneq G_2 \text{ means } \exists G_{iso} \subseteq G_2 : G_{iso} \cong G_1$$

Here, the partial order $\preceq$ on the embedding space is defined component-wise, meaning $\phi(G_1) \preceq \phi(G_2)$ if and only if $\phi_i(G_1) \leq \phi_i(G_2), \forall i \in [\![1, d]\!]$ such that $\phi = (\phi_1, \phi_2, \ldots, \phi_d)$. The partial order $\subsetneq$ on $\mathcal{G}(G)$ denotes subgraph isomorphism.

3.1.1 Subgraph Matching

In contrast to existing deep learning methods, Minomaly leverages transfer learning by using a single pretrained model for subgraph matching with order embedding. This eliminates the need for retraining on each dataset, as the model generalizes well to different graph structures generated synthetically during training using various random graph models including Erdős-Rényi graphs [**?**], Watts-Strogatz graphs [**?**], Barabási-Albert graphs [**?**], and Power Law Cluster graphs [**?**].

As illustrated in Figure 3, the trained model employs GNN with a multi-layer perceptron (MLP) for binary classification ($G_1 \subseteq G_2$). It takes two graphs, G_1 and G_2, as inputs. For the GNN architecture, we employ GraphSAGE [**?**] due to its sampling-based neighborhood aggregation, which enables scalable processing of large graphs by avoiding the full neighborhood expansion required by other GNN architectures. The model processes the graph structure through 8 layers of GraphSAGE-based graph convolutions with learnable skip connections [**?**]. A post-processing layer with linear transformations and Leaky ReLU activations produces the embeddings $\phi(G_1)$ and $\phi(G_2)$. These embeddings are concatenated and passed through an MLP to generate the output. The goal is to train the network to learn an order embedding function ϕ that respects its definition. To train the model, a set of positive and negative training instances are generated. For positive examples, we sample random subgraphs G_1^+ and G_2^+ from the graph G such that $G_1^+ \subseteq G_2^+$. For negative examples, we add enough

random nodes or edges to a subgraph G_1^- to generate G_2^-, or we can use any other valid random generation. The learning penalty proposed by [**?**] is adopted, and it is defined as

$$E(G_1, G_2) = \sum_{i \in [\![1,d]\!]} \| \max(0, \phi_i(G_1) - \phi_i(G_2)) \|^2$$

The total hinge loss is then given by:

$$\sum_{(G_1^+, G_2^+) \in \text{Positives}} E(G_1^+, G_2^+) + \sum_{(G_1^-, G_2^-) \in \text{Negatives}} \max\left(0, \alpha - E\left(G_1^-, G_2^-\right)\right) \quad (1)$$

where $\alpha > 0$ is the margin hyperparameter which controls the separation between penalties of positive and negative examples. Consequently, for positive pairs $G_1^+ \subseteq G_2^+$, the embeddings are trained to satisfy $\phi(G_1^+) \preceq \phi(G_2^+)$, as the penalty $E(G_1^+, G_2^+)$ will be zero with proper learning and large in order to be minimized during training. For negative pairs G_1^- and G_2^-, the violation $E(G_1^-, G_2^-)$ is trained to be at least α, ensuring separation from positive pairs and minimizing the overall loss.

Smooth Margin Loss

The hinge loss in Equation (1) suffers from a *dead gradient zone*: for any negative pair with $E(G_1^-, G_2^-) > \alpha$, the gradient is exactly zero and the model stops learning from it. With a small margin ($\alpha = 0.1$), most negative pairs quickly surpass this threshold early in training, causing the model to lose the learning signal that would push them further apart in the embedding space and improve the discriminability of the order embedding.

To address this, we replace the hard hinge $\max(0, x)$ with the **softplus** function $\text{sp}(x) = \log(1 + e^x)$, which is a smooth, everywhere-differentiable approximation satisfying $\text{sp}(x) \to \max(0, x)$ as $|x| \to \infty$, while maintaining a non-zero gradient $\text{sp}'(x) = \sigma(x) \in (0, 1)$ for all x. This yields the smooth margin loss:

$$\mathcal{L}_{\text{sp}} = \frac{1}{|\mathcal{P}|} \sum_{(G_1^+, G_2^+) \in \mathcal{P}} E(G_1^+, G_2^+) + \frac{1}{|\mathcal{N}|} \sum_{(G_1^-, G_2^-) \in \mathcal{N}} \log\left(1 + e^{\,\alpha - E(G_1^-, G_2^-)}\right) \quad (2)$$

where $\mathcal{P}$ and $\mathcal{N}$ denote the sets of positive and negative pairs, and the normalization by $|\mathcal{P}|$ and $|\mathcal{N}|$ makes the loss invariant to batch size.

This formulation preserves all the order-embedding properties of the original loss—positive pairs are still driven toward zero violation, and negative pairs are pushed beyond the margin—while providing continuous gradient signal to negative pairs even after they exceed α. The gradient magnitude for negatives decays smoothly as $\sigma(\alpha - E) \to 0$ for well-separated pairs, so the additional learning signal does not destabilize training. In practice, this leads to a better-calibrated separation between positive and negative violation distributions, improving the downstream frequency estimation and anomaly detection quality.

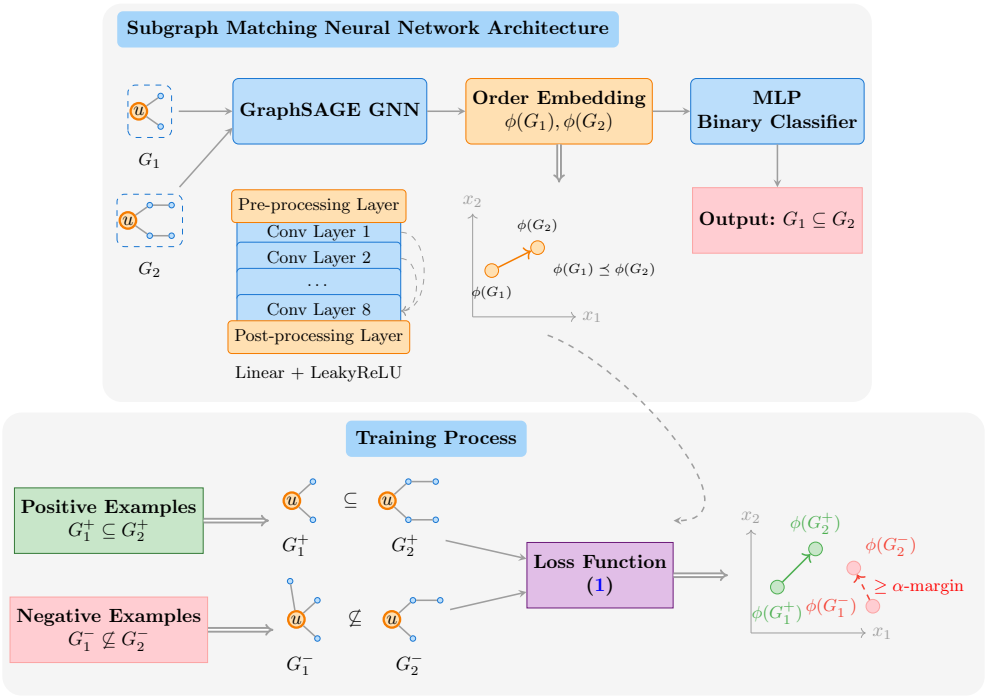


Fig. 3: Subgraph Matching Neural Network Architecture

Definition 6 (Anchor Node)**.** *For implementation purposes in the neural network architecture, we define the* ***anchor node*** *as a special node u from which the subgraph is defined.*

In subgraph matching, the anchor node plays a crucial role in guiding the GNN to focus on a localized neighborhood, ensuring consistency in subgraph extraction and embedding. By centering the subgraph around an anchor node, the GNN learns structural patterns relative to a fixed reference point, facilitating more robust and meaningful comparisons in the OES.

3.1.2 Frequency Count

The primary function of the subgraph matching model based on Order Embedding is to determine whether a given graph is a subgraph of another, enabling the quantification of its frequency within the entire graph. This is represented by any function $freq_G : \mathcal{G}(G) \to \mathbb{N}$, which measures the existence of its similar (isomorphic) subgraphs in G.

Building upon the order embedding framework, we now introduce our frequency measure $Freq_G$, which is based on matching subgraphs in the graph. It is defined as the number of graphs that contain a matching of G' ($|UP_G(G')|$). Our proposed measure is formalized as follows:

Definition 7 (Proposed Frequency Measure)**.** *For any $G' \in \mathcal{G}(G)$, we define:*

$$\begin{cases} UP_G(G') & = \{G_{up} \subseteq G \mid \exists G_{iso} \subseteq G_{up} : G_{iso} \cong G'\} \\ & = \{G_{up} \subseteq G \mid \phi(G') \preceq \phi(G_{up})\}, \\ Freq_G(G') & = |UP_G(G')|. \end{cases}$$

The correctness of frequency measures in pattern mining is traditionally evaluated through the anti-monotonicity property [**?**], a well-established criterion that ensures consistency in frequency-based anomaly detection.

Definition 8 (Anti-monotonicity Counting Property)**.** *$freq_G$ is anti-monotonic, if for all $G_1, G_2 \in \mathcal{G}(G)$:*

$$G_1 \subseteq G_2 \implies freq_G(G_2) \leq freq_G(G_1)$$

This implies that the frequency of a graph is never greater than the frequency of any of its subgraphs. Consequently, if a graph is infrequent, all graphs containing it are also infrequent.

Proposition 1. ***The proposed frequency measure $Freq_G$ is anti-monotonic.***

Proof. Let $G_1, G_2 \in \mathcal{G}(G)$ such that $G_1 = (S_1, E_1)$, $G_2 = (S_2, E_2)$ and $G_1 \subseteq G_2$.

- If $G_{up} \in UP_G(G_2)$, then $\exists G_{iso} \subseteq G_{up}$ such that $G_{iso} \cong G_2$.
- Since $G_1 \subseteq G_2$, $\exists G'_{iso} \subseteq G_{iso} \subseteq G_{up}$ such that $G'_{iso} \cong G_1$ (by restricting the bijection $f : S_2 \to T$ from Definition 3 to $S_1 \subseteq S_2$).
- Thus, $G_{up} \in UP_G(G_1)$, and we have $UP_G(G_2) \subseteq UP_G(G_1)$.

Taking cardinalities: $|UP_G(G_2)| \leq |UP_G(G_1)|$, i.e. $Freq_G(G_2) \leq Freq_G(G_1)$, therefore, *$Freq_G$ is anti-monotonic.* □

To estimate this frequency $Freq_G$ using the trained GNN model —since computing the exact $Freq_G$ via subgraph matching is NP-complete—, the approach leverages an efficient approximation. As illustrated in Figure 4, the method first decomposes the graph G into random subgraphs $\boldsymbol{RG} = \{G'_1, G'_2, \ldots, G'_k\}$ of different sizes from `min_neigh_size` to `max_neigh_size`, $\boldsymbol{k}$ must be sufficiently large. Subgraph sampling follows the tree sampling approach [**?**], which extracts connected neighborhoods by randomly selecting an anchor node and performing breadth-first traversal until the desired size is reached. These subgraphs are then processed through the GNN model to generate their respective order embeddings in order to construct the Order Embedding Space (OES) that is used for counting subgraph occurrences. The approach of subgraph matching is considered advantageous for these reasons:

- ***Generalizability*** — The model shows the generalization test performance of subgraph matching across real-world datasets, by training only on a general balanced synthetic dataset. On the other hand, the model only handles subgraph matching without considering features.
- ***OES geometric properties*** — Using the partial order properties of the embeddings, the frequency of a subgraph G' can be quantified by counting the

upper-right embeddings of RG graphs. Additionally, the anti-monotonicity property can be verified by how the space is organized according to graph size and order.

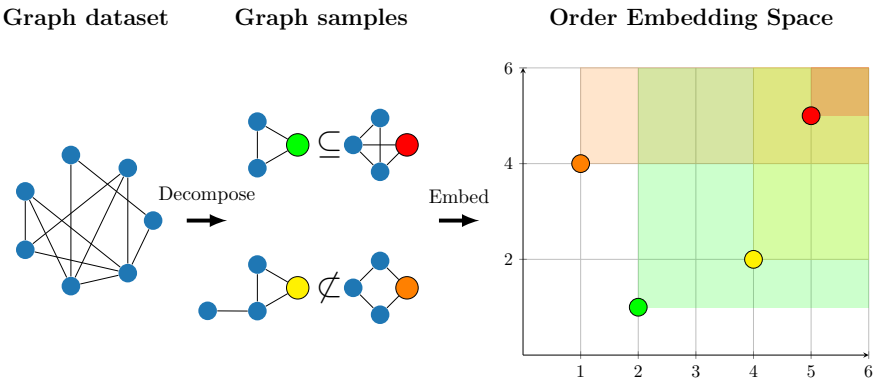


Fig. 4: Order Embedding Space construction process

After constructing the OES, the frequency of a subgraph G', $Freq_G(G')$, is determined by counting the number of the random embedded subgraph G'_i from $RG = \{G'_1, G'_2, \ldots, G'_k\}$ in the space, where $\phi(G') \preceq \phi(G'_i)$ as predicted by the model's output layer.

$$Freq_G(G') = \frac{|\{G'_i \in RG \mid \phi(G') \preceq \phi(G'_i)\}|}{k}$$

The embeddings $\phi(G'_i)$ correspond to the upper-right points of $\phi(G')$ in the geometric space OES (see Figure 5).

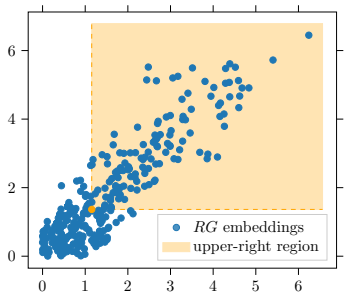
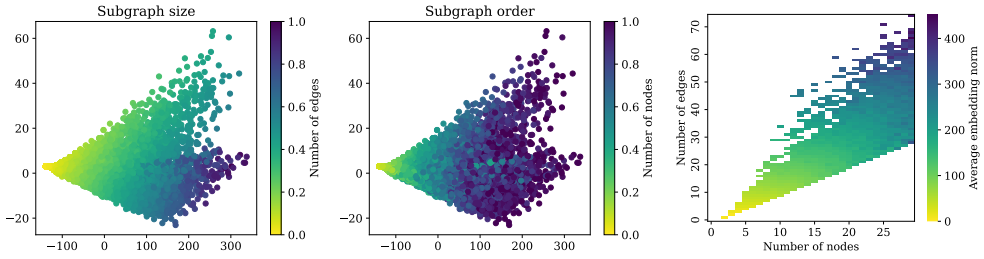


Fig. 5: Geometric visualization of the count prediction procedure in the OES

To experimentally verify the anti-monotonicity of the estimated $Freq_G$, we analyze the results shown in Figure 6. The trained GNN model, applied to the Cora dataset (see Table 3), organizes the space based on subgraph size (number of edges) and order (number of nodes), as larger graphs cannot be subgraphs of smaller ones.

This observation indicates that $Freq_G$ exhibits anti-monotonicity, supporting its correctness.



(a) PCA-based visualization (b) Heatmap

Fig. 6: (a) PCA visualization of order embeddings for random neighborhoods of 1–30 nodes from the Cora dataset. PCA preserves straight lines, indicating subgraph containment of a point lies within a quadrilateral region [**?**]. (b) The heatmap shows average embedding norms by graph size and order, highlighting structure-embedding relationships. Lighter shades are for smaller graphs, darker for larger ones by size or order.

3.2 Anomalous Structures

Defining and estimating the frequency of subgraphs, along with verifying its correctness through the anti-monotonicity property on node-anchored structures, provides a foundational step in identifying structural node anomalies based on the rarity of their neighborhoods. To achieve this, we formally propose our general definition of structural anomalies using subgraph frequency, without imposing any specific subgraph patterns as in [**? ?**]:

Definition 9 (Structural Anomaly)**.** *Let $u \in V$. The node u is a* ***structural anomaly*** *of G if and only if u belongs to an unexpected structure graph of u.*

- *We define an unexpected graph $G' = (V', E')$ in G as a rare subgraph in terms of graph frequency $freq_G : \mathcal{G}(G) \to \mathbb{N}$ that quantifies the existence of its similar (isomorphic) subgraphs in G:*

$$freq_G(G') < T_F \quad with \quad T_F > 0$$

- *All nodes $u' \in V'$ of the graph G' are structural anomalies.*

Given the estimated $Freq_G$ from the order embedding function ϕ and the OES, the goal of the search procedure is to identify rare node-anchored structures within the given graph. As illustrated in Figure 7, Minomaly detects anomalous small neighborhoods by iteratively expanding infrequent structures through the addition of neighboring nodes. Subgraphs that meet a certain frequency threshold T_F are identified as structural anomalies. The resulting graph represents the anomalies, demonstrating the interpretability of the method.

The approach adopts a greedy search (See Algorithm 1 and Figure 7d) for anomalous structures G', beginning from a starting node u and growing by iteratively selecting a candidate node u' from the current neighborhood frontier to obtain the most infrequent structure at each step. This process continues until one of the following conditions is met: $Freq_G(G') < T_F$, the maximum subgraph order (`max_steps`) is reached, or no significant change is observed (`max_no_change`). If the first condition is satisfied, the final small structure and all its nodes are identified as anomalies based on the final frequency, with the anomaly score defined as $1 - Freq_G(G')$.

Algorithm 1 Greedy Search for Anomalous Structures

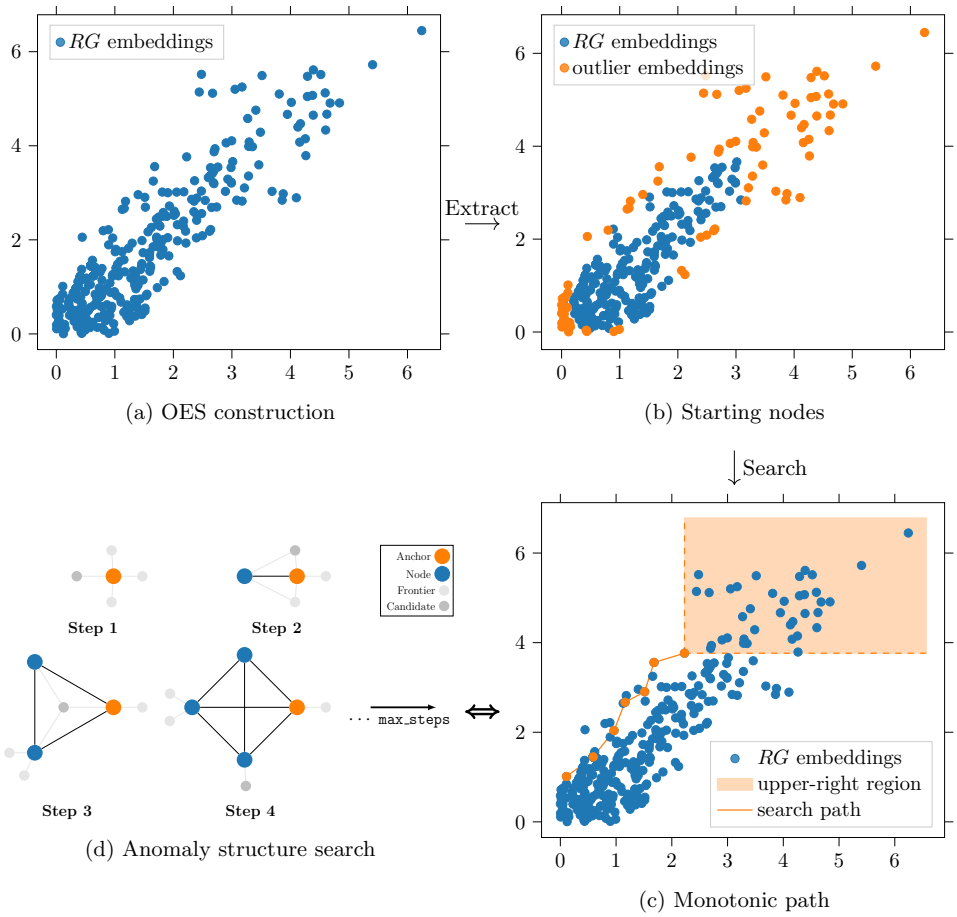
Require: Graph G, starting node u, threshold T_F, `max_steps`, `max_no_change`, `max_cands`
Ensure: Anomalous structure G', Anomaly score `score`

1: Initialize $G' \leftarrow \{u\}$, `steps` $\leftarrow 0$, `no_change_steps` $\leftarrow 0$, `score` $\leftarrow 0$
2: **while** `steps` $<$ `max_steps` **do**
3: `steps` $\leftarrow$ `steps` $+ 1$
4: Initialize $best\_freq \leftarrow \infty$, $G'' \leftarrow G'$
5: Sample a subset of `max_cands` candidate nodes from the neighborhood frontier of G'
6: **for** each sampled node v **do**
7: $G_{\text{temp}} \leftarrow G' \cup \{v\}$
8: **if** $Freq_G(G_{\text{temp}}) < best\_freq$ **then**
9: $best\_freq \leftarrow Freq_G(G_{\text{temp}})$
10: $G'' \leftarrow G_{\text{temp}}$
11: $G' \leftarrow G''$
12: **if** $Freq_G(G') < T_F$ **then**
13: Mark G' and all its nodes as anomalies
14: **break**
15: **if** $Freq_G(G')$ unchanged **then**
16: `no_change_steps` $\leftarrow$ `no_change_steps` $+ 1$
17: **else**
18: `no_change_steps` $\leftarrow 0$
19: **if** `no_change_steps` $\geq$ `max_no_change` **then**
20: **break**
21: `score` $\leftarrow 1 - Freq_G(G')$ **return** G', `score`

Note on Candidate Sampling: In Algorithm 1 (line 5), candidate nodes are sampled using Weighted Random Sampling. Weights are based on node visit frequency during neighborhood exploration, with frequently visited nodes receiving higher weights as they contribute to infrequent patterns. Other relevant sampling strategies could also be applied.

This search can be viewed as a monotonic walk in the search space of OES, which also demonstrates the preserved anti-monotonicity property due to the organized structure of the OES (See Figure 7c). If the resulting neighborhood G' of u is grown by the ordered sequence of subgraphs $G_u^{(1)} \subseteq G_u^{(2)} \subseteq \cdots \subseteq G_u^{(l)}$ such that $l \in [\![1, \texttt{max\_steps}]\!]$ and $G' = G_u^{(l)}$, a monotonic path of embedding search $\phi(G_u^{(1)}) \preceq \phi(G_u^{(2)}) \preceq \cdots \preceq \phi(G_u^{(l)})$ is observed, which in turn shows the anti-monotonicity of $Freq_G$:

$$Freq_G(G_u^{(l)}) \leq \cdots \leq Freq_G(G_u^{(2)}) \leq Freq_G(G_u^{(1)})$$



(a) OES construction

(b) Starting nodes

(c) Monotonic path

(d) Anomaly structure search

Fig. 7: OES construction and anomalous structure search procedures

The Algorithm 1 is not applied to all graph nodes, but to potential **starting nodes** (See Figure 7b) that are extracted from the OES in two ways:

(a) ***OES infrequent embedded subgraphs*** — The infrequent neighborhoods G'_i from $RG = \{G'_1, G'_2, \ldots, G'_k\}$ can be checked using the count prediction function $Freq_G(G'_i) < T_F$. The resulting anchors of these subgraphs are considered anomalous. Two types of neighborhoods can be selected: small infrequent ones, and larger ones that are infrequent due to the anti-monotonicity property, which may also contain smaller subgraphs contributing to their infrequency.

(b) ***OES frontier*** — Taking advantage of the geometric properties of OES, isolated and boundary points are more likely not to have upper-right points, which can be approximately detected by an outlier detection method like Isolation Forest [**?**]. These points are isolated due to the margin hyperparameter α, which separates them from the rest of the embeddings.

In addition, this method can be used to find anomalous structures of **node anomalies detected by other methods** that lack interpretations or have low precision by considering these anomalies as starting nodes.

3.3 Complexity Analysis

We analyze the time and space complexity of Minomaly. Let $n = |V|$ and $m = |E|$ denote the number of nodes and edges of G, k the number of random subgraphs in RG, d the embedding dimension, L the number of GNN layers, s the maximum neighborhood size (`max_neigh_size`), T the maximum search steps (`max_steps`), c the maximum candidates per step (`max_cands`), S the number of starting nodes, and B the embedding batch size.

OES Construction

Tree sampling generates k random neighborhoods, each of at most s nodes, via breadth-first frontier walk, in $O(k \cdot s)$ time. These neighborhoods are then embedded through the L-layer GNN. With learnable skip connections, the SAGEConv at layer i receives the concatenation of all $i{+}1$ previous representations as input, yielding an input dimension of $(i{+}1) \cdot d$. Since message passing at layer i performs a linear projection of each neighbor embedding—$O(|E_{\text{sub}}| \cdot (i{+}1) \cdot d^2)$—and the update step projects the concatenation of aggregated and self features—$O(|V_{\text{sub}}| \cdot (i{+}2) \cdot d^2)$—summing over all L layers on a subgraph of at most s nodes ($|E_{\text{sub}}| \leq s^2$) gives $O(s^2 \cdot L^2 \cdot d^2)$ per subgraph. Over k subgraphs:

$$T_{\text{OES}} = O\big(k \cdot s^2 \cdot L^2 \cdot d^2\big) \tag{3}$$

Since s, L, and d are bounded constants in practice ($s \leq 30$, $L = 8$, $d = 64$), this is effectively $O(k)$.

Starting Node Selection

The model-based outlier detection computes $Freq_G(G'_i)$ for each of the k neighborhoods by scanning all k cached embeddings. Each pairwise order-embedding check computes the violation $E(G_1, G_2) = \sum_{j=1}^{d} \|\max(0, \phi_j(G_1) - \phi_j(G_2))\|^2$ in $O(d)$ time, followed by the binary classifier in $O(1)$. For each of the k neighborhoods, this check is performed against all k embeddings in $\lceil k/B \rceil$ batches:

$$T_{\text{starting}} = O(k^2 \cdot d) \tag{4}$$

The Isolation Forest detection operates in $O(k \cdot d \cdot \log k)$, which is dominated by the model-based detection.

Greedy Search (Algorithm 1)

For each of the S starting nodes, the greedy search runs at most T steps. At each step, the algorithm evaluates up to c candidate nodes sampled from the neighborhood frontier. For each candidate, two operations are performed:

1. **Embedding**: a GNN forward pass on a subgraph of at most $s{+}1$ nodes, costing $O(s^2 \cdot L^2 \cdot d^2)$. All c candidates are batched in a single forward pass.
2. **Frequency computation**: scanning all k cached embeddings in $\lceil k/B \rceil$ batches, with $O(d)$ per pairwise comparison, yielding $O(k \cdot d)$ per candidate.

Since the frequency computation iterates over $\lceil k/B \rceil$ batches for each candidate while embedding is a single batched GPU pass, the frequency computation dominates. The total search complexity is:

$$T_{\text{search}} = O(S \cdot T \cdot c \cdot k \cdot d) \tag{5}$$

Two properties reduce the practical cost below this worst case. First, the anti-monotonicity (Proposition 1) guarantees that $Freq_G$ decreases monotonically along each search path, enabling early termination when $Freq_G(G') < T_F$, often well before T steps. Second, the `max_no_change` criterion prunes stagnating search paths, discarding candidates whose frequency has plateaued for a fixed number of iterations.

When `max_cands` is unrestricted (e.g., Cora), c equals the current frontier size, bounded by $O(\bar{\Delta} \cdot t)$ at step t, where $\bar{\Delta}$ is the average degree. For denser graphs, `max_cands` is capped to a constant (e.g., $c = 20$ for Amazon, $c = 10$ for Flickr), making the per-step cost independent of graph density.

Overall Time Complexity

The total time complexity of Minomaly is:

$$\boxed{T_{\text{total}} = O\left(\underbrace{k \cdot s^2 \cdot L^2 \cdot d^2}_{\text{OES construction}} + \underbrace{k^2 \cdot d}_{\text{Starting nodes}} + \underbrace{S \cdot T \cdot c \cdot k \cdot d}_{\text{Greedy search}} \right)} \tag{6}$$

In practice, the greedy search dominates, as confirmed by the experimental runtimes (e.g., 293s search vs. 22s OES construction for Cora). With fixed architectural constants (s, L, d), the complexity simplifies to $O(k^2 + S \cdot T \cdot c \cdot k)$. Note that $S = \rho \cdot n$ where ρ is the fraction of starting nodes ($\rho \in [17.9\%, 50.7\%]$ in our experiments), so the search cost scales linearly with both n and k.

Space Complexity

$$M_{\text{total}} = O(n + m + k \cdot d) \tag{7}$$

where $O(n{+}m)$ stores the input graph and $O(k{\cdot}d)$ stores the cached OES embeddings. The active beams occupy $O(S \cdot s)$, which is negligible. Crucially, this is *linear* in graph size, unlike reconstruction-based methods which require $O(n^2)$ memory for the adjacency reconstruction matrix $\hat{A} = \sigma(ZZ^\top)$ where $Z \in \mathbb{R}^{n \times d_h}$. This explains the out-of-memory failures of DOMINANT, AnomalyDAE, DONE, and AdONE on Flickr ($n = 89{,}250$), as shown in Table 4. Since k is a user-controlled parameter independent of n, Minomaly's memory footprint can be tuned to fit available resources.

Comparison with Baseline Methods

Table 1 summarizes the asymptotic complexity of all evaluated methods, where E_p denotes training epochs and d_h the hidden dimension.

Table 1: Asymptotic complexity comparison of graph anomaly detection methods

Method	**Time**	**Space**
GCNAE	$O(E_p \cdot L \cdot m \cdot d_h + E_p \cdot n^2 \cdot d_h)$	$O(n^2 + n \cdot d_h)$
DOMINANT	$O(E_p \cdot L \cdot m \cdot d_h + E_p \cdot n^2 \cdot d_h)$	$O(n^2 + n \cdot d_h)$
AnomalyDAE	$O(E_p \cdot L \cdot m \cdot d_h + E_p \cdot n^2 \cdot d_h)$	$O(n^2 + n \cdot d_h)$
DONE / AdONE	$O(E_p \cdot n^2 \cdot d_h)$	$O(n^2 + n \cdot d_h)$
CONAD	$O(E_p \cdot L \cdot m \cdot d_h + E_p \cdot n^2 \cdot d_h)$	$O(n^2 + n \cdot d_h)$
Minomaly	$O(S \cdot T \cdot c \cdot k \cdot d)$	$O(n + m + k \cdot d)$

Reconstruction-based methods incur $O(n^2)$ space for the dense adjacency reconstruction, making them impractical for large graphs. Minomaly's space grows linearly with k, a user-controlled parameter independent of n. Its time complexity depends on the product $S \cdot k$ (starting nodes times OES size), both of which scale controllably with graph size through the sampling parameters and the starting node selection strategy.

4 Experiments

In this section, we evaluate Minomaly through extensive experiments. We begin by outlining the experimental protocol, which includes a description of the datasets used, an introduction to the representative node anomaly detection methods for comparison, and a discussion of the evaluation metrics commonly employed in anomaly detection. Next, we present the results, comparing the performance of our approach with the baseline methods. Finally, the performance study examines the impact of dataset characteristics and hyperparameters, along with an evaluation of interpretability, scalability, and the efficiency of our search method.

4.1 Experimental Protocol

This section describes the experimental setup used to evaluate Minomaly. Our experiments were conducted on a Linux machine with Python 3.12.2, CUDA 12, and an NVIDIA RTX A2000 GPU (12 GB). We rely on SPMiner [**?**] for subgraph mining. Minomaly was implemented using PyTorch 2.4, Torch Geometric 2.6, and DeepSNAP, while the benchmarked methods were implemented following [**?**]. We performed our benchmark following the protocol outlined in [**?**], using the hyperparameter grid and datasets provided in the paper.

We set the Minomaly hyperparameters shown in Table 2 as configurations (C_1, A_1, F_1) for each dataset, fixing the OES dimension to $d = 64$, `min_neigh_size` to 1, and `max_neigh_size` to 30. Additionally, we utilized 8 threads for the search phase. `batch_size` for Minomaly refers to the number of embedded *RG* subgraphs processed in a single batch.

These hyperparameters, are interpretable and manually tuned to improve results. This allows refinement after each run by filtering user-relative positives, expanding search spaces, or reducing computational resources, without random hyperparameter search.

The subgraph matching OES model of Minomaly, pretrained on synthetic graphs with margin $\alpha = 0.1$, achieves 95% accuracy in determining whether one graph is a subgraph of another with linear time complexity [**?**]. It was used for processing all the benchmarked datasets.

Table 2: Minomaly Hyperparameters Configurations

Parameter	$\mathbf{C_1}$	$\mathbf{A_1}$	$\mathbf{F_1}$
Dataset	**Cora**	**Amazon**	**Flickr**
$\boldsymbol{k}$	10,000	50,000	150,000
$\boldsymbol{T_F}$	$\frac{35}{2708} \approx 0.013$	$\frac{150}{13752} \approx 0.011$	$\frac{1000}{89250} \approx 0.011$
`max_steps`	7	11	5
`max_no_change`	5	3	3
`max_cands`	All	20	10
`batch_size`	1,000	10,000	10,000

4.1.1 Datasets

To evaluate Minomaly, we use the following datasets, which are provided by [**? ?**] and have been used in previous works [**? ? ? ? ?**]:

- **Cora** [**?**] : The Cora dataset is a network of machine learning papers, represented as a directed graph that illustrates the citation relationships between papers.
- **Amazon** [**?**] : The Amazon Computers network consists of nodes representing products, with edges indicating that two products are frequently purchased together.
- **Flickr** [**?**] : The Flickr dataset is a network of online images from Flickr, with links formed between images that share common properties.

Table 3: Dataset Statistics

Dataset	Nodes	Edges	Avg. Degree	Anomalies
Cora	2,708	11,060	4.1	70 (5.1%)
Amazon	13,752	515,042	37.2	350 (5.0%)
Flickr	89,250	933,804	10.5	2,240 (4.9%)

The table 3 summarizes the key characteristics of the three datasets used in our experiments. It presents the number of nodes and edges for each dataset, the average node degree (which reflects the typical connectivity), and the number (and percentage) of anomalies detected.

These datasets offer diverse graph structures—from the sparse connectivity of Cora to the high interconnectivity of Amazon, with Flickr balancing large size and moderate

connectivity. The consistent anomaly ratio (around 5%) across all datasets facilitates a fair evaluation of the performance and robustness of various anomaly detection methods.

4.1.2 Compared Methods

The following state-of-the-art methods were selected as baselines to benchmark Minomaly's performance, representing diverse deep learning approaches to GAD. For the Cora and Amazon datasets, we employed full-batch training with 400 epochs, while the larger Flickr dataset required mini-batch processing with batch size 64 and reduced training to just 1 epoch with 3-hop neighborhood sampling. Hidden dimensions of $\{32, 64, 128, 256\}$ were explored for all three datasets. Across all methods, we maintained consistent regularization with dropout rates $\{0, 0.1, 0.3\}$, learning rates $\{0.01, 0.05, 0.1\}$, and weight decay of 0.01. We tuned the balance weight α $\{0.2, 0.5, 0.8\}$ between reconstruction and structural losses, following the protocol of [**?**]. Hyperparameters were randomly selected over 20 trials, and we report the performance as the mean and standard deviation across these trials.

- **GCNAE** [**? ?**]: An autoencoder using GCNs as encoder and decoder to learn node embeddings from graph structure and attributes. Anomaly scores are based on reconstruction error.
- **DOMINANT** [**?**]: Uses a GCN encoder and dual decoders - one for node attribute reconstruction and another for graph structure reconstruction. The final anomaly score combines both reconstruction errors to capture different aspects of abnormality.
- **AnomalyDAE** [**?**]: Similar to DOMINANT but employs two separate autoencoders (AEs). Its structural encoder uniquely processes both adjacency information and node features simultaneously, while its attribute decoder leverages both structural and attribute embeddings for reconstruction.
- **DONE** [**?**]: Uses separate MLP-based autoencoders for structure and attributes, with a unified optimization objective that simultaneously learns node embeddings and anomaly scores through a joint loss function.
- **AdONE** [**?**]: Extends DONE by incorporating adversarial learning with a discriminator that works to align structural and attribute embeddings in the latent space.
- **CONAD** [**?**]: Leverages graph augmentation and contrastive learning based on human prior knowledge of structural and contextual anomalies. Anomalies are scored using a combination of contrastive loss and graph reconstruction error.

4.1.3 Evaluation Metrics

To comprehensively assess anomaly detection performance, we employed standard binary classification metrics that are particularly relevant for imbalanced datasets typical in anomaly detection. In the context of anomaly detection, true positives (TP) represent correctly identified anomalies, false positives (FP) are normal nodes incorrectly flagged as anomalies, true negatives (TN) are correctly identified normal nodes, and false negatives (FN) are anomalies that were missed. These metrics evaluate both

the ranking quality of anomaly scores and the classification performance at various thresholds:

- **Precision**: It measures the proportion of true positives among the predicted positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall**: (or True Positive Rate TPR) It measures the proportion of true positives among the actual positives.

$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN}$$

- **F1 Score**: It is the harmonic mean of precision and recall. Balances recall and precision, providing a single measure of overall performance.

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **False Positive Rate** (FPR): It measures the proportion of false positives among the actual negatives.

$$\text{FPR} = \frac{FP}{FP + TN}$$

- **Average Precision (AP)**: It summarizes the precision-recall curve as the weighted mean of precision achieved at each threshold, with the increase in recall from the previous threshold used as the weight.

$$\text{AP} = \sum_n (R_n - R_{n-1}) \times P_n$$

  where P_n and R_n are the precision and recall at the nth threshold.
- **Area Under the ROC Curve (AUC)**: It measures the entire two-dimensional area underneath the ROC curve, which plots TPR against FPR at different classification thresholds. The AUC ranges from 0 to 1, where 0.5 represents random classification performance, and 1.0 represents perfect classification.

4.2 Results

Table 4 presents a comprehensive comparison between Minomaly and state-of-the-art GAD methods across three datasets of varying size and complexity. Minomaly demonstrates superior performance across all evaluation metrics, achieving the best average ranks in F1 (1.0), AP (1.0), Recall (1.0), and Precision (1.0), while ranking second only on Cora's AUC metric (average rank 1.3 overall).

On the Cora dataset, Minomaly achieves outstanding F1, AP, Recall, and Precision scores, significantly outperforming all competitors, with impressive margins of 50.8 percentage points in F1 and 51.9 percentage points in AP compared to the second-best method. For the Amazon dataset, Minomaly demonstrates even greater dominance, surpassing all alternatives across every metric, with particularly notable improvements

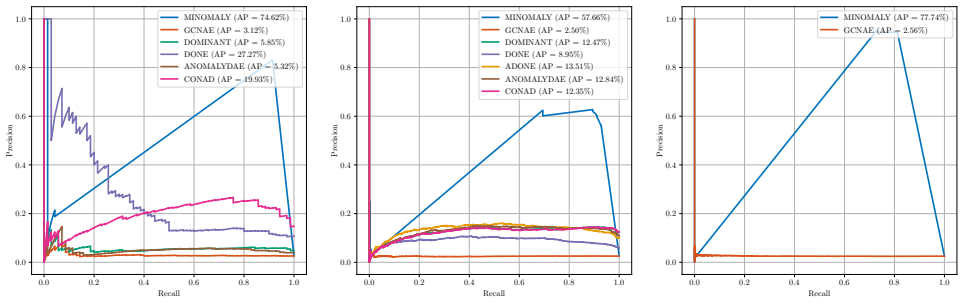
in F1 (69.6% versus 21.7% for the next best) and AP (57.7% versus 12.2%). Perhaps most significantly, Minomaly is the only method capable of effectively handling the large-scale Flickr dataset, where most competing approaches fail due to memory limitations. Only GCNAE could execute on Flickr but performed poorly (49.9% AUC versus Minomaly's 90.5%).

Table 4: Comparison of AUC, F1, AP, Recall, and Precision across methods

Dataset	Model	AUC	F1	AP	Recall	Precision
Cora	Minomaly	95.3 ± 0.0 (Rank: 2.0)	**87.1** ± 0.0 (Rank: 1.0)	**74.6** ± 0.0 (Rank: 1.0)	**91.4** ± 0.0 (Rank: 1.0)	**83.1** ± 0.0 (Rank: 1.0)
	GCNAE	52.5 ± 0.0 (Rank: 7.0)	5.3 ± 0.0 (Rank: 7.0)	3.1 ± 0.0 (Rank: 7.0)	12.9 ± 0.0 (Rank: 7.0)	3.3 ± 0.0 (Rank: 7.0)
	DOMINANT	92.1 ± 9.3 (Rank: 3.0)	32.3 ± 10.6 (Rank: 3.0)	19.9 ± 6.5 (Rank: 4.0)	78.7 ± 25.9 (Rank: 3.0)	20.3 ± 6.7 (Rank: 3.0)
	DONE	90.1 ± 5.9 (Rank: 5.0)	20.2 ± 11.0 (Rank: 5.0)	22.7 ± 18.8 (Rank: 2.0)	49.2 ± 26.9 (Rank: 5.0)	12.7 ± 6.9 (Rank: 5.0)
	AdONE	91.1 ± 5.0 (Rank: 4.0)	25.7 ± 11.1 (Rank: 4.0)	16.3 ± 7.1 (Rank: 5.0)	62.6 ± 27.0 (Rank: 4.0)	16.2 ± 7.0 (Rank: 4.0)
	AnomalyDAE	81.7 ± 11.8 (Rank: 6.0)	16.7 ± 14.1 (Rank: 6.0)	11.7 ± 8.3 (Rank: 6.0)	40.6 ± 34.3 (Rank: 6.0)	10.5 ± 8.9 (Rank: 6.0)
	CONAD	**95.4** ± 1.2 (Rank: 1.0)	36.3 ± 3.9 (Rank: 2.0)	22.3 ± 2.8 (Rank: 3.0)	88.5 ± 9.5 (Rank: 2.0)	22.9 ± 2.4 (Rank: 2.0)
Amazon	Minomaly	**95.4** ± 0.0 (Rank: 1.0)	**69.6** ± 0.0 (Rank: 1.0)	**57.7** ± 0.0 (Rank: 1.0)	**92.9** ± 0.0 (Rank: 1.0)	**55.7** ± 0.0 (Rank: 1.0)
	GCNAE	49.0 ± 0.0 (Rank: 7.0)	3.7 ± 0.0 (Rank: 7.0)	2.5 ± 0.0 (Rank: 7.0)	9.1 ± 0.0 (Rank: 7.0)	2.3 ± 0.0 (Rank: 7.0)
	DOMINANT	91.4 ± 0.1 (Rank: 2.0)	21.6 ± 0.2 (Rank: 3.0)	12.2 ± 0.1 (Rank: 2.0)	53.1 ± 0.4 (Rank: 3.0)	13.5 ± 0.1 (Rank: 3.0)
	DONE	87.3 ± 4.0 (Rank: 5.0)	17.8 ± 6.8 (Rank: 5.0)	11.7 ± 4.4 (Rank: 5.0)	43.9 ± 16.8 (Rank: 5.0)	11.2 ± 4.3 (Rank: 5.0)
	AdONE	85.5 ± 5.6 (Rank: 6.0)	13.2 ± 7.2 (Rank: 6.0)	9.3 ± 4.0 (Rank: 6.0)	32.6 ± 17.8 (Rank: 6.0)	8.3 ± 4.5 (Rank: 6.0)
	AnomalyDAE	89.7 ± 5.8 (Rank: 4.0)	21.7 ± 3.4 (Rank: 2.0)	12.0 ± 2.0 (Rank: 4.0)	53.4 ± 8.4 (Rank: 2.0)	13.6 ± 2.1 (Rank: 2.0)
	CONAD	91.3 ± 0.1 (Rank: 3.0)	21.5 ± 0.2 (Rank: 4.0)	12.2 ± 0.1 (Rank: 3.0)	53.1 ± 0.4 (Rank: 4.0)	13.5 ± 0.1 (Rank: 4.0)
Flickr	Minomaly	**90.5** ± 0.0 (Rank: 1.0)	**87.3** ± 0.0 (Rank: 1.0)	**77.7** ± 0.0 (Rank: 1.0)	**81.2** ± 0.0 (Rank: 1.0)	**94.3** ± 0.0 (Rank: 1.0)
	GCNAE	49.9 ± 0.1 (Rank: 2.0)	4.4 ± 0.1 (Rank: 2.0)	2.6 ± 0.0 (Rank: 2.0)	10.9 ± 0.2 (Rank: 2.0)	2.7 ± 0.0 (Rank: 2.0)
	Others	Out of GPU Memory				
Avg. Rank	Minomaly	**1.3**	**1.0**	**1.0**	**1.0**	**1.0**
	GCNAE	5.3	5.3	5.3	5.3	5.3
	DOMINANT	2.5	3.0	3.0	3.0	3.0
	DONE	5.0	5.0	3.5	5.0	5.0
	AdONE	5.0	5.0	5.5	5.0	5.0
	AnomalyDAE	4.0	4.0	5.0	4.0	4.0
	CONAD	2.0	3.0	3.0	3.0	3.0

From the Precision-Recall curves in Figure 8, we observe low Precision@k across all methods (precisions in left side region), which corresponds to low-frequency scores detected as anomalies for Minomaly. This is attributed to the existence of some unlabeled structures that are rarer than the labeled ones, as shown in the interpretations (See Figure 9b). DONE achieves high Precision@k on Cora due to its restrictive anomaly definition but fails to maintain precision on other datasets and for higher recalls, unlike our frequency-based method. This demonstrates that using frequency

as a measure of abnormality is more effective in identifying true anomalies, surpassing reconstruction-loss methods that struggle with rare patterns and fail to capture anomalous patterns across different contexts, as mentioned in Section 2.



(a) Cora (b) Amazon (c) Flickr

Fig. 8: Precision-Recall curves for different models.

4.3 Performance Study

In this section, we provide a deep comprehensive analysis of Minomaly's performance characteristics beyond comparative benchmarking. First, we evaluate the effectiveness of our starting node selection strategy, quantifying its computational benefits and impact on detection accuracy by examining how the size k of RG affects performance on Flickr. Next, we analyze how the `max_cands` parameter balances effectiveness and efficiency across different levels of graph connectivity. Additionally, we present interpretable visualizations of detected anomalous structures and their frequency evolution. Finally, we conduct a detailed hyperparameter study, examining how TF and `max_steps` influence detection performance across various evaluation metrics, revealing clear trends that facilitate practical parameter tuning.

To analyze the impact of the size k of RG on detection quality and its significance in covering patterns within large datasets, we extend Table 2 by introducing two additional configurations, F_2 and F_3, for the Flickr dataset, as shown in Table 5.

Table 5: Minomaly Flickr Hyperparameters

Parameter	**Run F_1**	**Run F_2**	**Run F_3**
Dataset	**Flickr**	**Flickr**	**Flickr**
k	150,000	200,000	250,000
T_F	$\frac{1000}{89250} \approx 0.011$	$\frac{1000}{89250} \approx 0.011$	$\frac{1000}{89250} \approx 0.011$
`max_steps`	5	5	5
`max_no_change`	3	3	3
`max_cands`	10	10	10
`batch_size`	10,000	12,500	12,500

The results corresponding to each hyperparameter configuration (C_1, A_1, F_1, F_2, F_3) are detailed in Table 6.

Table 6: Minomaly Results

Result	**Run C_1**	**Run A_1**	**Run F_1**	**Run F_2**	**Run F_3**
Dataset	Cora	Amazon	Flickr	Flickr	Flickr
Starting nodes	$\frac{1120}{2708} \approx 41.4\%$	$\frac{6977}{13752} \approx 50.7\%$	$\frac{15988}{89250} \approx 17.9\%$	$\frac{20057}{89250} \approx 22.5\%$	$\frac{24074}{89250} \approx 27.0\%$
Positive starting nodes	$\frac{67}{70} \approx 95.7\%$	$\frac{300}{350} \approx 85.7\%$	$\frac{1535}{2240} \approx 68.5\%$	$\frac{1736}{2240} \approx 77.5\%$	$\frac{1898}{2240} \approx 84.7\%$
AUC	95.25%	95.40%	90.52%	93.56%	94.06%
AP	74.62%	57.66%	77.73%	85.28%	85.82%
Recall	91.43%	92.86%	81.21%	87.19%	88.21%
Precision	83.12%	55.65%	94.34%	94.99%	93.73%
Search time (s)	293	2770	2521	4098	5690
OES build time (s)	22	118	542	1021	1335

Impact of Starting Nodes Strategy and RG Size

Table 6 demonstrates the quantitative impact of our starting nodes selection strategy on both computational efficiency and detection performance. By selecting a subset of nodes as starting points based on methods (a) *OES infrequent embedded subgraphs* and (b) *OES frontier* detection, described in 3.2—rather than examining all nodes—Minomaly reduces computational complexity while maintaining high detection quality. For the Cora and Amazon datasets, the algorithm processes only 41.4% and 50.7% of nodes respectively, while for the larger Flickr dataset, the processing requirement decreases to between 17.9% and 27.0% of nodes.

The precision of these selected starting nodes is quantifiably high, with 95.7% of starting nodes in Cora and 85.7% in Amazon being true positives. In Flickr, as parameter k increases from 150,000 in Run F_1 to 250,000 in Run F_3, we observe a corresponding increase in the proportion of positive starting nodes from 68.5% to 84.7%. This improved node selection directly correlates with enhanced detection performance: AUC increases from 90.52% to 94.06%, AP from 77.73% to 85.82%, and recall from 81.21% to 88.21% across the three Flickr runs.

This performance improvement introduces a computational trade-off: as the proportion of starting nodes in Flickr increases, search time increases from 2521s to 5690s. The consistently high precision (93.73%-94.99% across all Flickr runs) confirms that the approach minimizes unnecessary searches in regions prone to false positives, thus optimizing the balance between computational cost and detection effectiveness. These results validate that the strategic selection of starting nodes from anomalous neighborhoods effectively reduces the search space while preserving the algorithm's node anomaly detection capabilities.

Impact of `max_cands` Parameter

Table 6 and Table 3 together reveal important insights about the `max_cands` hyperparameter's effect on performance and computational efficiency across datasets with varying connectivity characteristics. This parameter, as shown in Algorithm 1, limits the number of candidate nodes considered from the neighborhood frontier during the greedy search process.

For the Cora dataset, with its relatively low average degree (4.1), all frontier nodes were considered during the search (`max_cands` = "All"). This complete frontier exploration was computationally feasible due to Cora's limited connectivity and starting nodes, resulting in the shortest search time (293s) among the three datasets despite examining all possible candidates.

In contrast, for the Amazon dataset with its high connectivity (average degree 37.2), restricting `max_cands` to 20 was necessary to maintain computational tractability. Despite this significant frontier sampling restriction, the model achieved an AUC of 95.40% and a recall of 92.86%, demonstrating that examining only a smaller subset of the densely connected frontier can yield strong detection performance.

For Flickr, the largest dataset with moderate connectivity (average degree 10.5), `max_cands` was further restricted to 10. The model achieved AUC scores between 90.52% and 94.06% across different runs. The search time increased with the number of starting nodes examined (from 2521s to 5690s), but remained manageable despite the dataset's scale.

The experimental findings suggest that even restrictive frontier sampling (examining only 10-20 candidates) can achieve strong anomaly detection performance on large, dense graphs, as the search still identifies structurally meaningful anomalous patterns despite not examining all possible expansions at each step. This is due to the `max_neigh_size` size of the random subgraphs that was limited to 30, so the OES was meaningful enough to extract the properties of the diversity of small neighborhoods, that enabled also parallel frequency calculations and significantly reduces the number of candidate nodes (`max_cands`) to grow anomalous node structures.

Extracted Anomalous Patterns

Tables 7, 8, and 9 present the predicted anomalous pattern groups for the three datasets. After identifying anomalous structures in each dataset, we applied Weisfeiler-Lehman (WL) graph hashing to group structurally isomorphic patterns together; all nodes of these discovered structures are predicted as anomalous. Each table shows representative graphs for these pattern groups, where **red nodes** indicate true anomalous nodes and **green nodes** indicate true normal nodes from the ground-truth. The **Count** field shows how many times each pattern appears in the dataset, while **A/N** denotes the composition of true anomalous versus true normal nodes within each representative pattern.

The pattern extraction results demonstrate Minomaly's strong interpretability advantage, enabling users to understand and validate detected anomalies through visual structural analysis. For Cora (Table 7), the dominant pattern 7-0 (Count: 31, A/N: 7/0) reveals a fully connected clique structure consisting entirely of anomalous nodes, accounting for the majority of detected anomalies and contributing to the

method's high precision (83.12%) on this dataset. Flickr (Table 9) exhibits similar behavior with patterns 7-0, 7-1, and 7-2 (Count: 375, 49, and 6 respectively) showing pure anomalous structures, supporting the excellent precision (93.73%-94.99%) observed in Table 6. In contrast, Amazon (Table 8) displays more complex pattern diversity with 10 distinct groups, where patterns like 11-1, 11-2, and 11-4 through 11-9 contain only normal nodes (A/N: 0/11) despite being structurally complex and similar to anomalous patterns, explaining the lower precision (55.65%) while maintaining excellent recall (92.86%). This visual pattern interpretation capability allows domain experts to manually filter false positives by examining structural characteristics and node compositions, effectively transforming the anomaly detection task into an interpretable pattern recognition problem. The visualization enables users to quickly identify which patterns merit further investigation versus those that can be safely filtered, significantly enhancing the practical utility of the detection results beyond raw performance metrics.

Table 7: Extracted anomalous patterns for Cora dataset

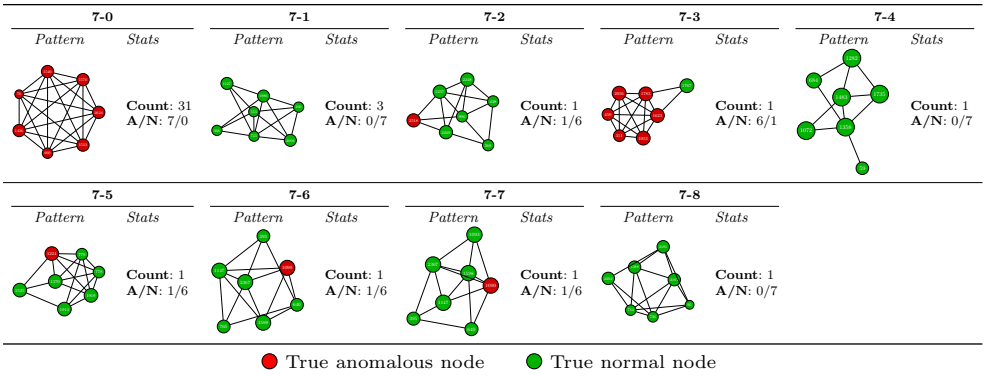


Table 8: Extracted anomalous patterns for Amazon dataset

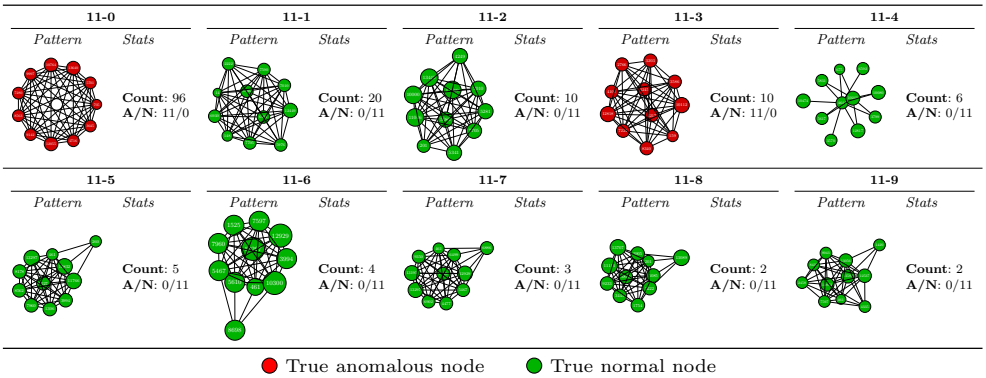
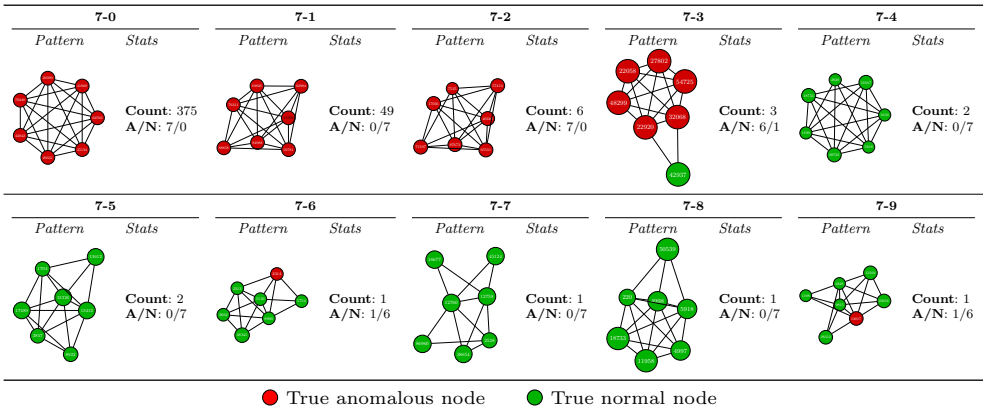
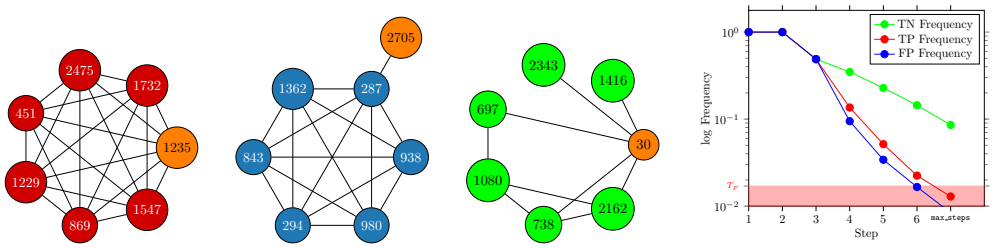


Table 9: Extracted anomalous patterns for Flickr dataset



Interpretations

While Minomaly demonstrates strong overall performance across datasets (Table 6), Amazon's precision (55.65%) is notably lower than that of Cora (83.12%) and Flickr (93.73%-94.99%). This discrepancy can be attributed to two primary factors revealed through our pattern interpretation analysis. Figure 9a, 9b and 9c illustrate examples of these interpretations. First, Amazon contains numerous structural patterns that are labeled as normal in the ground truth but exhibit strong structural similarities to true anomalous patterns. Second, nodes connected to anomalous structures are labeled as normal in the ground truth and can be labeled as anomalies in certain contexts. Despite these precision challenges, the method maintains excellent recall (92.86%) on Amazon, demonstrating its effectiveness at identifying the complete set of true anomalies.

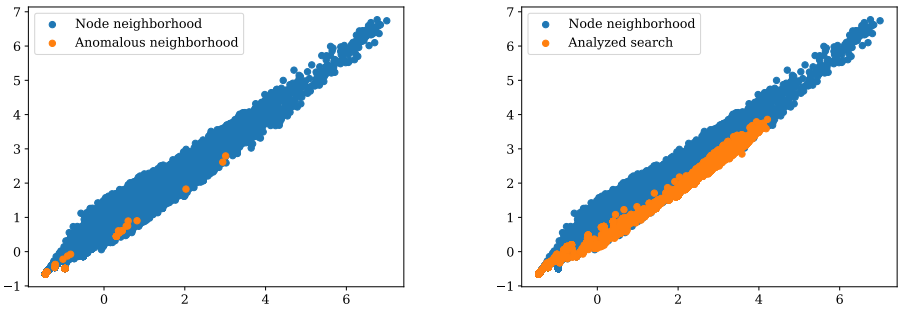


(a) True positive (b) False positive (c) True negative (d) Frequency changes

Fig. 9: Minomaly interpretation results on the Cora dataset, illustrating three examples: an anomalous structure (red), a normal one (green), and a falsely predicted node connected to an anomalous structure (blue), along with their frequency change curves.

Figure 10 visually demonstrates the effectiveness of the method in identifying anomalous regions by step-walking over the embedded anomalous neighborhoods,

which contributes to achieving high recall (92.86%) on the all datasets. Additionally, the overall high precision observed across datasets indicates that the approach effectively filters out true positive nodes from the large visualized searched region.



(a) True embedded anomalous neighborhoods (b) Anomalous neighborhoods search

Fig. 10: This figure shows the true anomalous neighborhood embeddings and their corresponding search analysis by the algorithm in the OES of Cora, visualized in two dimensions.

T_F *and* `max_steps` *Hyperparameters Study*

We study the effect of T_F and `max_steps` on AUC, AP, Recall, and Precision metrics, evaluating the anomaly detection framework on the Cora and Amazon, Flickr datasets (Figures 11, 12 and 13). The ranges of hyperparameters for each dataset are summarized in Table 10.

Table 10: Hyperparameter ranges for different datasets

Dataset	**Cora**	**Amazon**	**Flickr**
Total nodes	2,708	13,752	89,250
T_F range (unnormalized)	$[\![1, 40]\!]$	$\{10, 20, \ldots, 200\}$	$\{100, 200, \ldots, 1400\}$
max_steps range	$[\![1, 11]\!]$	$[\![1, 21]\!]$	$[\![1, 11]\!]$

T_F and `max_steps` control the threshold used to filter anomalous nodes from the search results, as shown in Figure 9d. Thus, it is important to study their effect on detection performance.

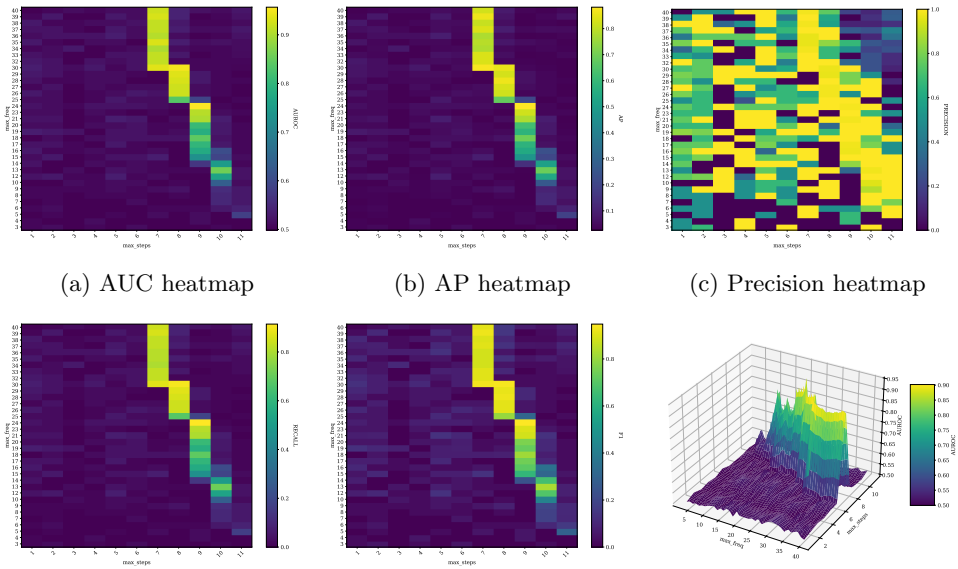
The heatmaps indicate that Minomaly is highly sensitive to these hyperparameters, and performance can be improved only by tuning them, as they serve as key thresholds for the framework (See Figures 11, 12, 13 and 9d).

Since AUC, AP, Recall, and F1 metrics share similar heatmaps, it is sufficient to use only AUC for manual tuning, as it is a reliable metric for this task. In contrast, the baseline methods that rely on reconstruction thresholds typically yield suboptimal results for the other metrics, despite having good AUC (See Table 4), because their reconstruction thresholds are independent of the hyperparameters, and choosing the right reconstruction threshold is another task to consider.

We also observe that precision is generally high across the entire search grid and shows little sensitivity to the hyperparameters, as illustrated in the heatmaps 11c, 12c and 13c, but it decreases for higher values of T_F and `max_steps`. This aligns with our focus on detecting rare small node neighborhoods.

The AUC surfaces for both datasets are relatively smooth, with a few local maxima, as shown in Figures 11f, 12f and 13f. The AUC heatmap shows optimal hyperparameters of $T_F = \frac{24}{2708} \approx 0.88\%$ and `max_steps` $= 9$ for Cora, $T_F = \frac{150}{13752} \approx 1.1\%$ and `max_steps` $= 11$ for Amazon, and $T_F = \frac{1000}{89250} \approx 1.1\%$ and `max_steps` $= 5$ for Flickr. These values are close to the used benchmark parameters in Table 2 and are within the region of interest. The AUC heatmap for Cora (Figure 11a), Amazon (Figure 12a) and Flickr 13a) show that high scores are achieved in a relatively continuous linear region of T_F and `max_steps`. Thus, finding the optimal values is straightforward. For small continuous values of T_F and `max_steps`, performance is high and stable, but it decreases outside these regions.

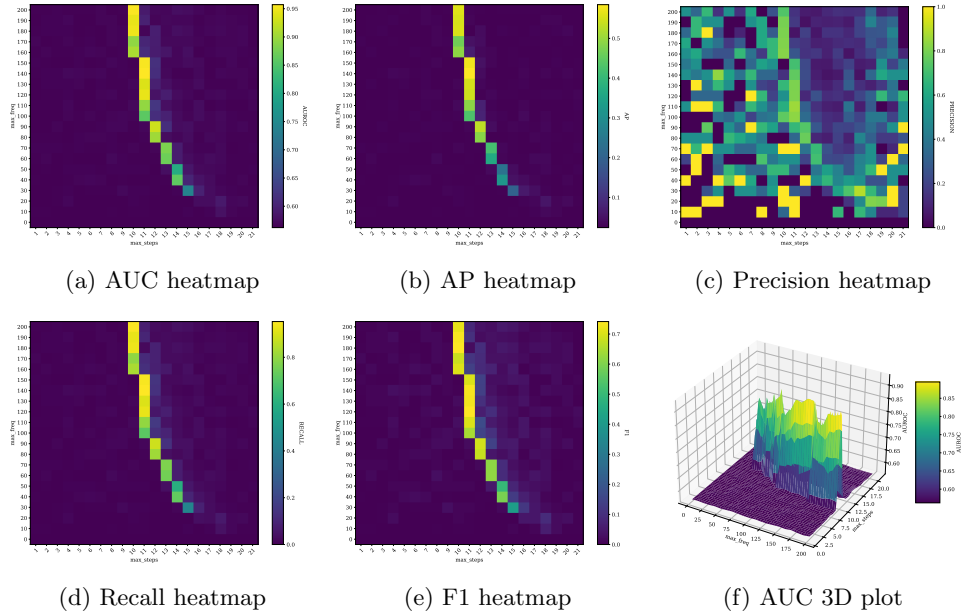
Finding these regions is not difficult: the frequency threshold T_F is typically small since anomalies are rare and can be determined from the dataset characteristics. `max_steps` is also small, as we focus on small neighborhoods, followed by manual tuning based on the interpretations. Alternatively, a faster automatic solution without varying the parameters in the greedy search is to predefine the search region by setting sufficiently high values for `max_steps` and T_F, then execute Minomaly with these values. The frequency curves will be generated and filtered for each frequency `freq` $< T_F$ and step `step` $<$ `max_steps`. The choice of optimal hyperparameters from `freq` and `step` is then left to the user based on the interpretations.



(a) AUC heatmap (b) AP heatmap (c) Precision heatmap

(d) Recall heatmap (e) F1 heatmap (f) AUC 3D plot

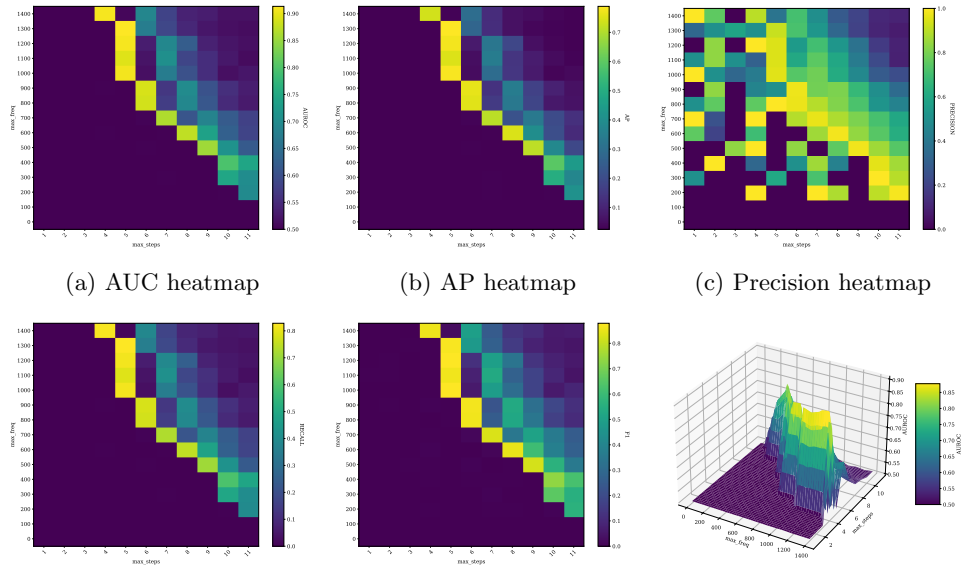
Fig. 11: T_F (y-axis) and `max_steps` (x-axis) parameter analysis on the Cora dataset.



(a) AUC heatmap (b) AP heatmap (c) Precision heatmap

(d) Recall heatmap (e) F1 heatmap (f) AUC 3D plot

Fig. 12: T_F (y-axis) and `max_steps` (x-axis) parameter analysis on the Amazon dataset.



(a) AUC heatmap (b) AP heatmap (c) Precision heatmap

(d) Recall heatmap (e) F1 heatmap (f) AUC 3D plot

Fig. 13: T_F (y-axis) and `max_steps` (x-axis) parameter analysis on the Flickr dataset.

5 Discussion

After presenting the results, we now highlight the key insights from our experiments. Minomaly not only outperforms existing methods but also provides a deeper understanding of structural anomaly detection in graphs.

Shifting from Reconstruction to Frequency-Based Detection

Minomaly's performance advantages across datasets validate our fundamental hypothesis that frequency-based anomaly detection captures structural abnormalities more effectively than reconstruction-based approaches. While reconstruction methods must compress graph information into lower-dimensional embeddings before reconstruction, potentially losing subtle structural differences, Minomaly directly measures structure rarity with minimal information loss. Traditional approaches often produce false positives or miss anomalies by directly using reconstruction error magnitude as their core measure of abnormality—assuming difficult-to-reconstruct structures are inherently anomalous. In contrast, Minomaly leverages a rich set of embeddings from multiple subgraphs and uses GNN-based comparisons to precisely quantify structure frequency without relying on a single reconstruction outcome. This approach provides a more deterministic and interpretable measure of abnormality compared to the often ambiguous nature of reconstruction errors. The conceptual shift proves particularly valuable on the Amazon dataset, where Minomaly achieves an F1 score of 69.6% compared to 21.7% for the next best competitor, demonstrating superior discrimination between truly anomalous and normal structural patterns.

Efficient and Scalable Search Strategy

Minomaly's selective processing of starting nodes (41.4% of Cora nodes, 50.7% of Amazon nodes, and 17.9%-27.0% of Flickr nodes) significantly reduces computation complexity without compromising detection quality. The high precision of these starting nodes (95.7% in Cora, 85.7% in Amazon) shows that we effectively identify regions containing anomalies. Unlike competing methods that suffer from memory limitations due to neighborhood explosion when reconstructing node representations, Minomaly remains scalable even on large graphs like Flickr (achieving 90.5%-94.1% AUC). This scalability comes from our frequency-based approach using a sufficiently k embedded random small graphs RG to cover diverse graph patterns, avoiding the memory-intensive operations that cause other methods to fail on larger datasets.

Interpretable Parameter Space

The analysis of T_F and `max_steps` reveals that Minomaly's hyperparameters have clear semantic meaning with predictable effects on performance. The continuous regions of high performance in Figures 11-13 suggest that finding effective configurations is straightforward compared to deep learning approaches. This interpretability facilitates practical deployment and tuning in real-world scenarios. Similarly, the `max_cands` parameter effectively balances exploration thoroughness and computational complexity, as evidenced by strong performance on Amazon (95.40% AUC) despite examining only 20 candidates per step in this highly connected graph.

Integrated Detection and Explanation

Unlike other methods that explain results afterward, Minomaly unifies detection and explanation into a coherent process. The frequency curves and anomalous structures in Figure 9 and Tables 7-9 provide transparent rationales for anomaly flags, making results more actionable for domain experts. Figure 10 visualizes how the search process effectively navigates the embedding space to identify anomalous regions, demonstrating that this integrated approach successfully captures meaningful anomalous structures.

These findings collectively demonstrate that Minomaly's frequency-based approach offers a robust foundation for structural anomaly detection that addresses fundamental limitations in current methods. By directly quantifying node structure rarity, selectively exploring the graph, and providing transparent interpretations, our method achieves superior performance while maintaining computational feasibility for large-scale applications. The clear interpretability of both the method and its results makes it particularly suitable for practical deployment in domains where understanding anomalies is as important as detecting them.

6 Conclusion

We introduced **Minomaly**, an unsupervised framework for detecting node structural anomalies in static graphs using subgraph mining and Graph Neural Networks. Our work formalizes structural anomaly detection through subgraph mining with theoretical foundations via the anti-monotonicity property, while generalizing structural node anomaly detection across multiple contexts.

Minomaly applies order embeddings to graph anomaly detection, leveraging GNNs to efficiently encode graph structures. By exploiting the geometrical properties of the Order Embedding Space, our approach significantly reduces verification overhead during anomalous structure search. We developed a computationally efficient greedy search algorithm that enables understandable graph anomaly detection by identifying and interpreting unusual structures.

Our comprehensive experiments demonstrate Minomaly's superior performance across all metrics while maintaining scalability on large datasets like Flickr. Unlike reconstruction-based methods, our approach uses frequency as a measure of abnormality with interpretable and controllable hyperparameters that can be manually tuned, avoiding memory issues while delivering better performance. Future work will incorporate node attributes for contextual anomalies, develop more robust node candidate sampling strategies to optimize the anomalies search for highly connected graphs, and extend the framework to dynamic graphs with temporal order embeddings and graph-level anomaly detection. Enhancing the state-of-the-art in structural anomaly detection is fundamental to addressing contextual anomalies due to their intrinsically related nature.